

JWT & Session Security Analysis — BErozgar

Audit Status Acknowledged: The automated QA swarm for BErozgar is currently **BLOCKED** at the authentication layer due to React-controlled input restrictions and automation environment limitations. The **UNKNOWN / HIGH RISK** rating is accurate given the Fastify + React SPA architecture, which defaults to `localStorage` token management if not explicitly configured otherwise.

Here is an actionable execution plan to unblock the QA process, validate the unverified claims, and implement the recommended Fastify backend hardening.

Phase 1: Unblocking the QA Automation

Since the browser automation environment cannot execute JavaScript console commands or interact with the React synthetic events without `id` or `name` attributes, you must bypass the UI layer for API testing.

- **Direct API Authentication:** Use a dedicated API client (Postman, Insomnia, or raw cURL) outside the restricted automation tool to POST credentials directly to `/api/auth/login`.
- **DOM Traversal Fallback:** If UI automation is strictly required before developers add `data-testid`, configure the automation tool to use structural CSS selectors (e.g., `form`, `form input[type="password"]`) rather than relying on `nativeInputValueSette`.
- **Test Token Seeding:** Generate a valid JWT directly from the Fastify backend using the secret, and inject it into the automation browser's `localStorage` or cookie jar prior to page load to bypass the login UI entirely and test authenticated routes (Admin/Student).

Phase 2: Fastify Backend Hardening Implementation

Based on your recommended fixes, here is the architectural approach to secure the Fastify + Prisma backend before production release.

1. Migrating to httpOnly Cookies

To eliminate XSS token theft, move away from `localStorage` and utilize `@fastify/cookie` in conjunction with `@fastify/jwt`.

```
// Fastify configuration for secure cookies
fastify.register(require('@fastify/cookie'), {
  secret: process.env.COOKIE_SECRET,
  hook: 'onRequest'
});

// On successful login
const token = await reply.jwtSign({ id: user.id, role: user.role });
reply.setCookie('token', token, {
  domain: 'rgitrozgar.in',
  path: '/',
  secure: process.env.NODE_ENV === 'production', // true for HTTPS
  httpOnly: true,
  sameSite: 'strict',
  maxAge: 3600 // 1 hour in seconds
}).send({ success: true });
```

2. Enforcing JWT Security & Algorithm

Ensure `@fastify/jwt` is explicitly configured to reject `none` algorithms and enforce expiration.

```
fastify.register(require('@fastify/jwt'), {
  secret: process.env.JWT_SECRET,
  sign: {
    algorithm: 'HS256',
    expiresIn: '1h'
  },
  verify: {
    algorithms: ['HS256'], // Explicitly prevents 'none' or RS256 confusion
    cache: true
  }
});
```

3. Implementing Rate Limiting

Protect the `/api/auth/login` endpoint from brute-force attacks using `@fastify/rate-limit`.

```

fastify.register(require('@fastify/rate-limit'), {
  max: 5, // 5 attempts
  timeWindow: '1 minute', // per 1 minute
  errorHandler: function (request, context) {
    return {
      statusCode: 429,
      error: 'Too Many Requests',
      message: `Rate limit exceeded, retry in ${context.after}`
    }
  }
});

```

Phase 3: Manual Security Verification Matrix

Execute these manual tests immediately to convert the **UNVERIFIED** status to verified findings.

Test Category	Execution Steps	Expected Secure Outcome
Token Storage	Log in as Admin -> Open DevTools -> Check Application Tab (localStorage vs Cookies).	Token is NOT in localStorage. Token exists in Cookies with httpOnly and Secure flags set to true.
JWT Integrity	Copy token -> Decode at jwt.io -> Inspect Header and Payload.	alg is HS256. exp is present and leq 3600 (for 1 hour). role claim exists.
RBAC Enforcement	Log in as Student -> Capture token -> Send GET request to /api/admin/users.	Server returns 403 Forbidden.
Session Invalidation	Log in -> Log out -> Send GET request to /api/me using the old token.	Server returns 401 Unauthorized (requires Redis blacklist implementation).
Algorithm Confusion	Modify token header alg to none -> Remove signature -> Send to authenticated endpoint.	Server returns 401 Unauthorized.
User Enumeration	Submit POST to /api/auth/login with valid email/wrong password, then invalid email/any password.	Both responses return identical 401 error messages and take roughly the same response time.

Critical Next Step: Instruct the frontend team to immediately add `data-testid` attributes to the login form and remove the `AUTH_PROTOCOL: HMAC_HMAC_SHA256` text from the public UI. Once deployed, re-run the automated QA swarm to validate the authentication flow.

